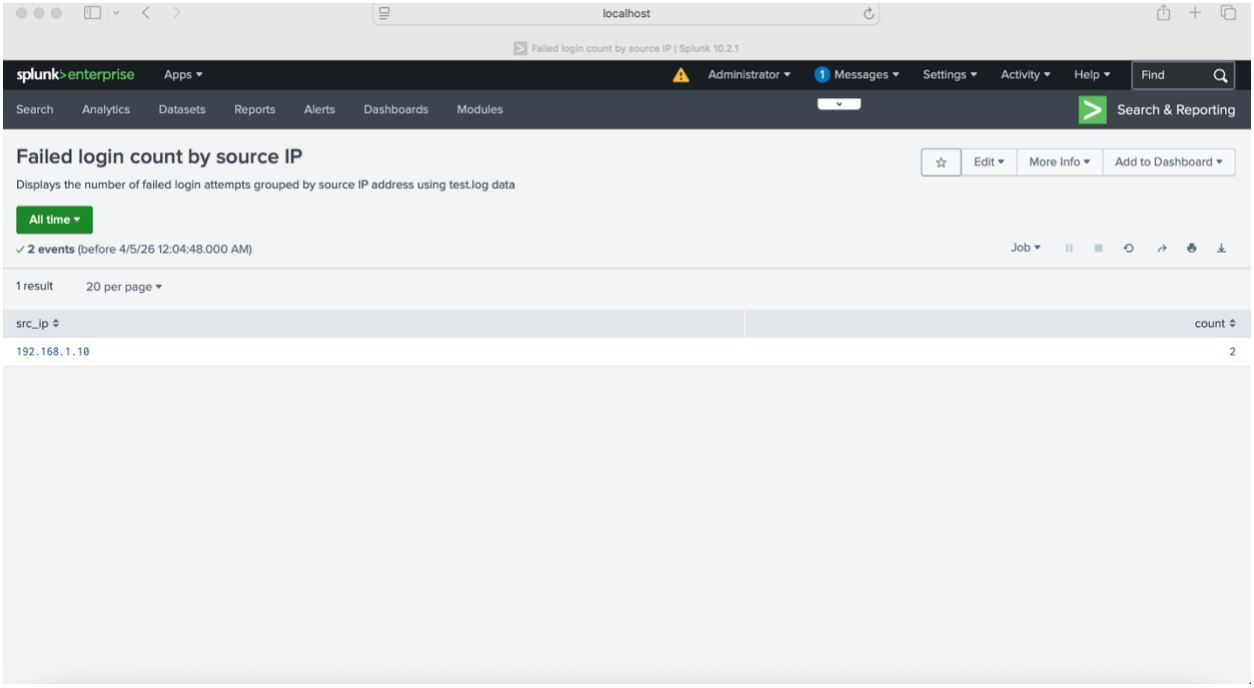
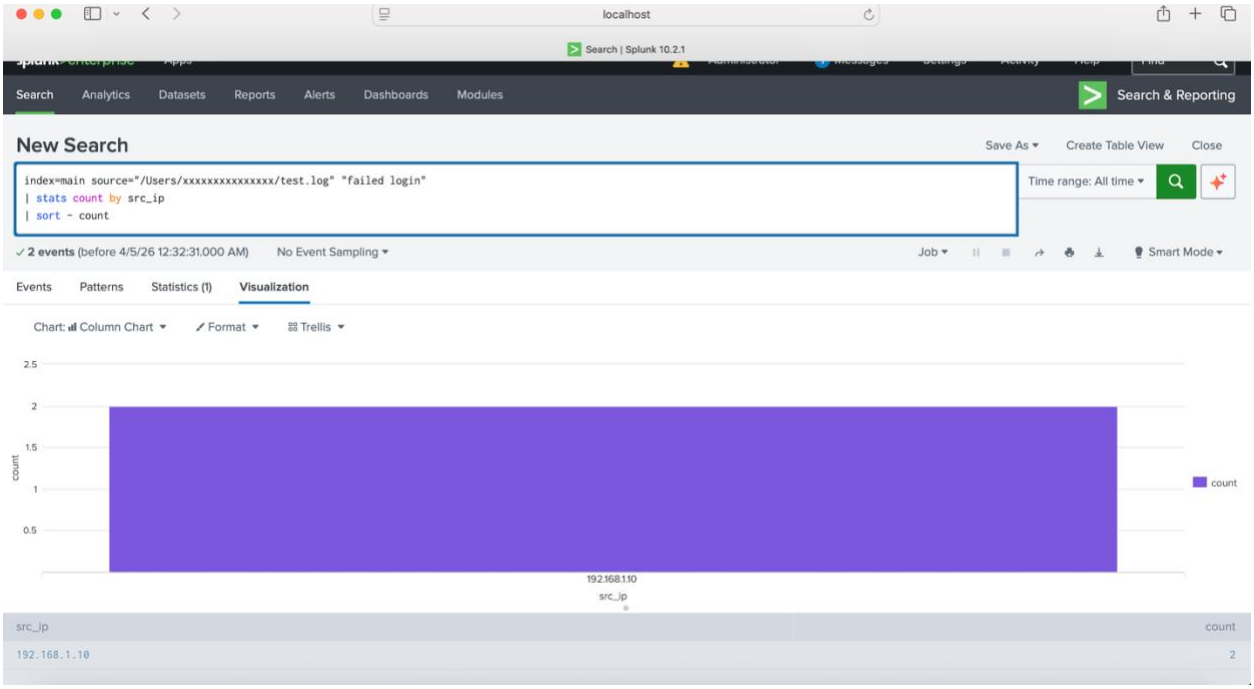
Splunk SOC Lab: 3-Phase Detection & Monitoring Workflow

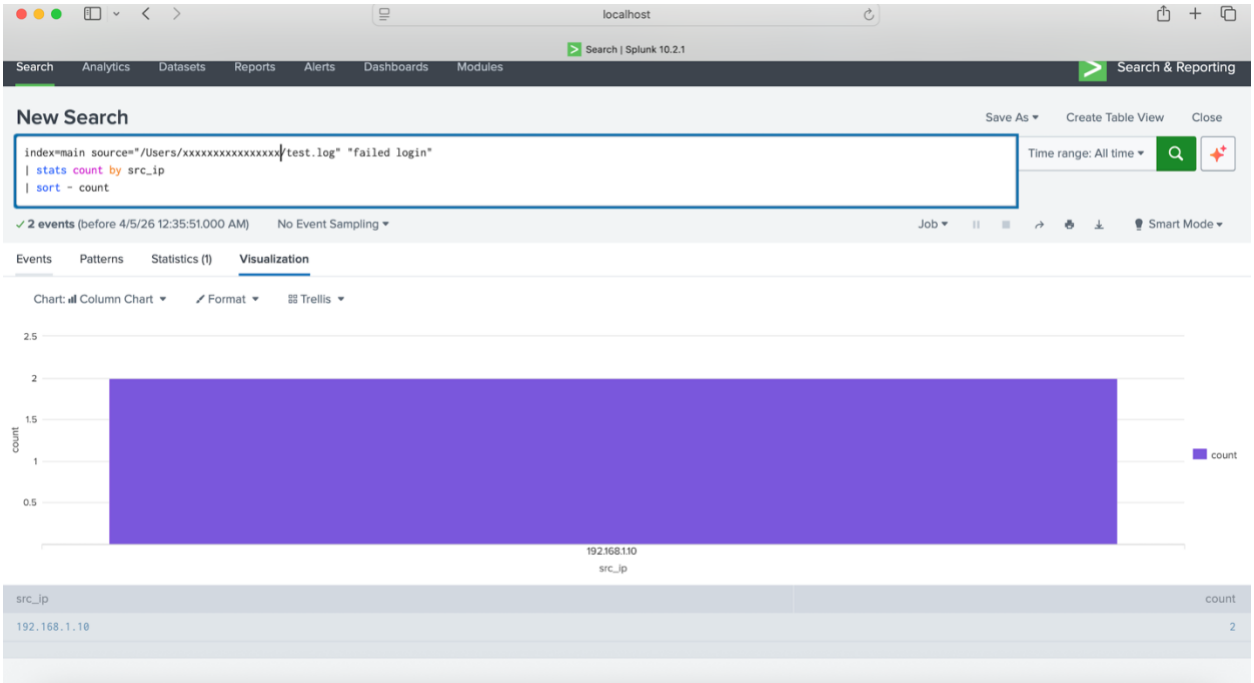
In this step, a search query was created in Splunk to identify failed login attempts within the dataset (**test.log**) and count how many times each source IP address appeared. Instead of repeatedly running the same query manually, the search was saved as a report titled **"Failed Login Count by Source IP."** In Splunk, a report represents a saved search that preserves the query, time range, and configuration, allowing it to be reused efficiently without re-entering the SPL each time. This approach reflects real-world SOC workflows, where analysts convert frequently used detection queries into reusable reports to streamline monitoring, support dashboards, and enable alerting. By saving this report, the analysis becomes persistent, scalable, and ready for integration into further security monitoring processes such as visualization and automated detection.

So, I use search query and turned it into a Report, and splunk saved it for me. What a Report Actually Is (Core Concept)
A Report in Splunk = Saved Search. It contains my SPL query, my time range, and my settings.





In this step, the saved report was transformed into a visual representation and integrated into a dashboard to simulate real-world security monitoring. The query results, which display the count of failed login attempts by source IP address, were converted into a bar chart to make patterns and high-risk activity easier to identify at a glance. This visualization was then added as a panel to a Classic Splunk dashboard titled "Security Monitoring Dashboard." By doing this, the analysis moved from manual querying to continuous visual monitoring, allowing security analysts to quickly detect suspicious behavior such as repeated failed login attempts from specific IP addresses. This step demonstrates how raw log data and queries can be operationalized into an interactive monitoring interface, which is a core function of SIEM platforms in a SOC environment.



In this step, an alert was created in Splunk to automate the detection of suspicious login activity based on the previously developed query. The alert, titled **"Brute Force Detection – Failed Logins by IP,"** was configured as a scheduled alert using a cron-based interval to continuously monitor incoming log data. The trigger condition was set to activate whenever the number of results exceeds zero, meaning that any detected failed login attempt would generate an alert. As part of the alert configuration, the action **"Add to Triggered Alerts"** was enabled, allowing the event to be recorded and reviewed within Splunk's alert management system. This implementation demonstrates how static analysis can be transformed into automated detection logic, enabling real-time monitoring of potential brute force attacks. It reflects a core function of SIEM platforms in a SOC environment, where analysts rely on alerts to identify and respond to security incidents efficiently.

Alert Configuration Details

The following alert was created to simulate automated detection of suspicious login activity within the Splunk environment:

- **Alert Title**:
  Brute Force Detection – Failed Logins by IP
- **Alert Type**:
  Scheduled (Cron-based execution)
- **Schedule**:
  Runs automatically at regular intervals using a cron expression (e.g., every 5 minutes)
- **Search Query**:

- • **Time Range**:
  Last 5 minutes
- • **Trigger Condition**:
  The alert is triggered when the number of search results is greater than zero
  (i.e., at least one failed login attempt is detected)
- • **Trigger Actions**:
  Add to Triggered Alerts

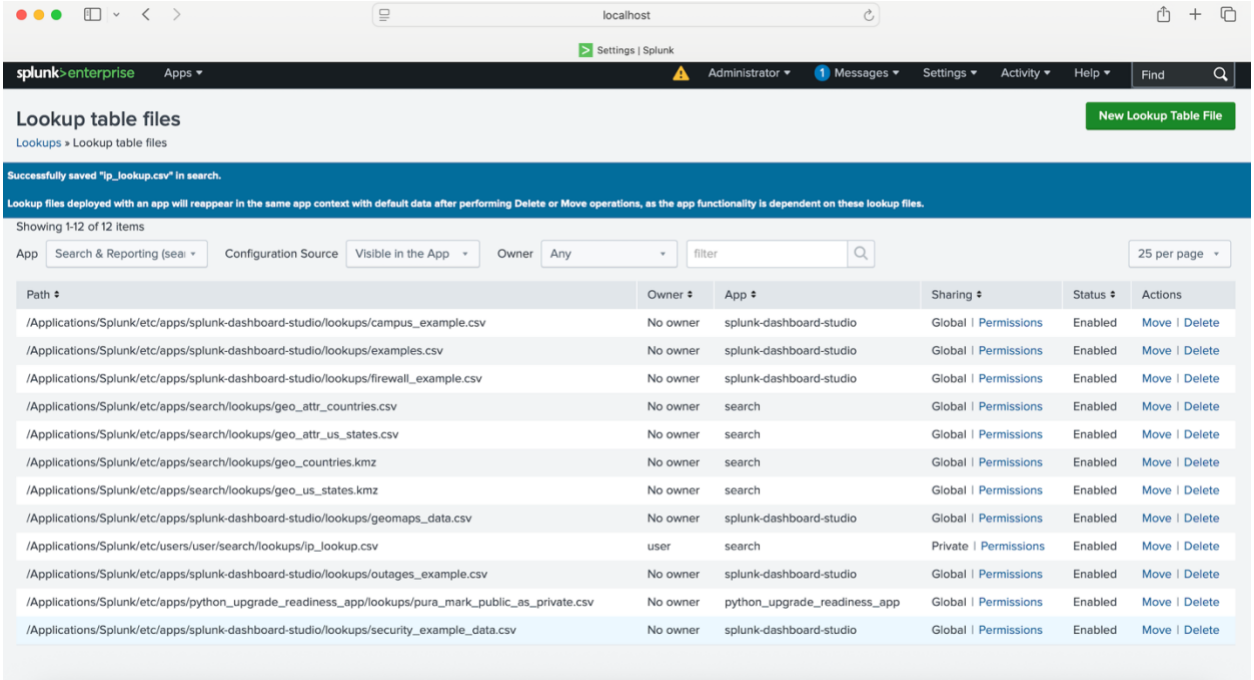
This alert simulates a basic brute-force detection mechanism by identifying repeated failed login attempts from source IP addresses. By automating this detection logic, the system continuously monitors incoming log data and generates alerts whenever suspicious activity is observed. This reduces the need for manual analysis and enables faster identification of potential security threats, which is a critical function of SIEM platforms in a SOC environment.

Creating csv lookup

In this step, a CSV lookup file named *ip_lookup.csv* was created locally to support data enrichment within the Splunk environment. The file contains a simple mapping between source IP addresses and descriptive labels such as *"Brute Force Attacker," "Suspicious Activity,"* and *"Internal Test System."* This lookup table serves as an external data source that can be used to add contextual meaning to raw log data during analysis. Instead of viewing IP addresses as isolated values, the lookup enables the analyst to classify and interpret activity more effectively. Creating this file represents the foundational step in implementing data enrichment, which is a critical capability in SIEM platforms for improving visibility, prioritization, and decision-making during security investigations.

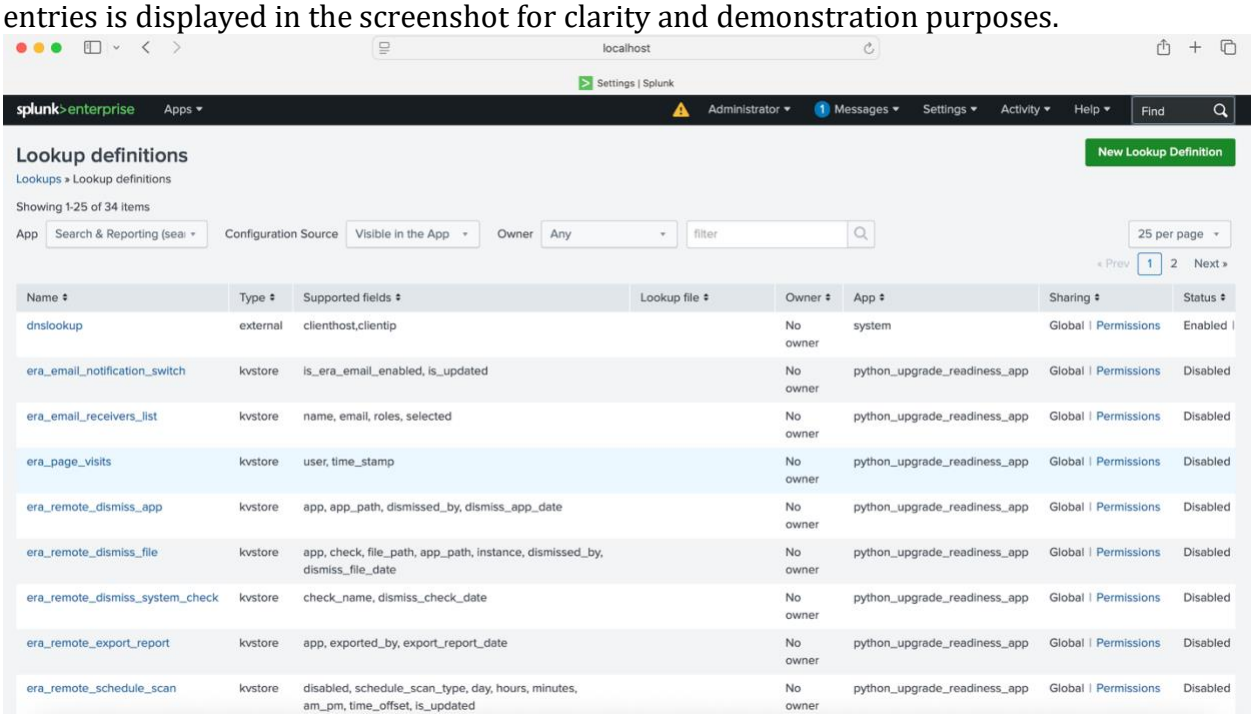
```
MacBookPro:~ xxxxxxxxxxxxxxxx$ cat ip_lookup.csv
src_ip,label
192.168.1.10,Brute Force Attacker
192.168.1.20,Suspicious Activity
192.168.1.30,Internal Test System
MacBookPro:~xxxxxxxxxxxxxxxx$
```

In this step, the CSV lookup file (*ip_lookup.csv*) was uploaded into Splunk through the **Settings → Lookups → Lookup table files** interface and saved within the *search* app, making it available for use in data enrichment and correlation with indexed log data.



Create Lookup Definition

In this step, a lookup definition was created in Splunk by navigating to **Settings → Lookups → Lookup definitions**, and add new where a new file-based lookup was configured using the uploaded *ip_lookup.csv* file and mapped on the `src_ip` field, enabling Splunk to match incoming log data with the lookup table and apply contextual enrichment during searches. Although the lookup definition returned a total of 34 matching results, only a subset of 9 entries is displayed in the screenshot for clarity and demonstration purposes.

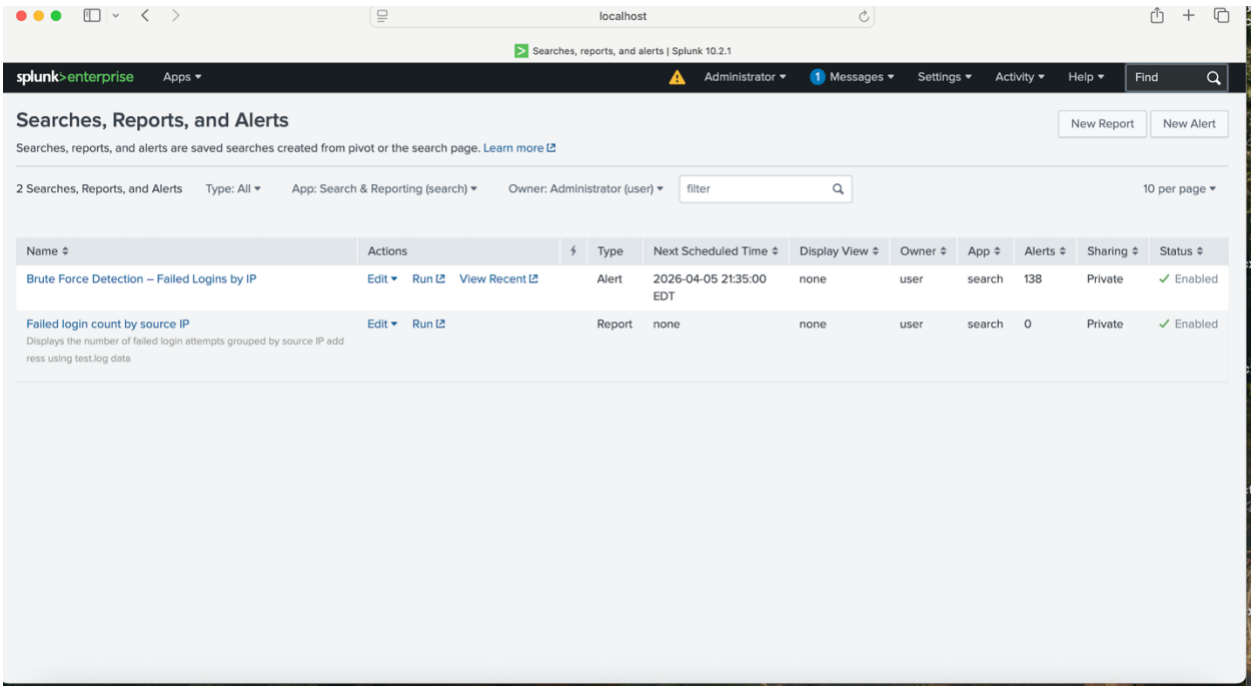


Apply Lookup in SPL (Data Enrichment)

I run this inquiry into Search and Reporting.

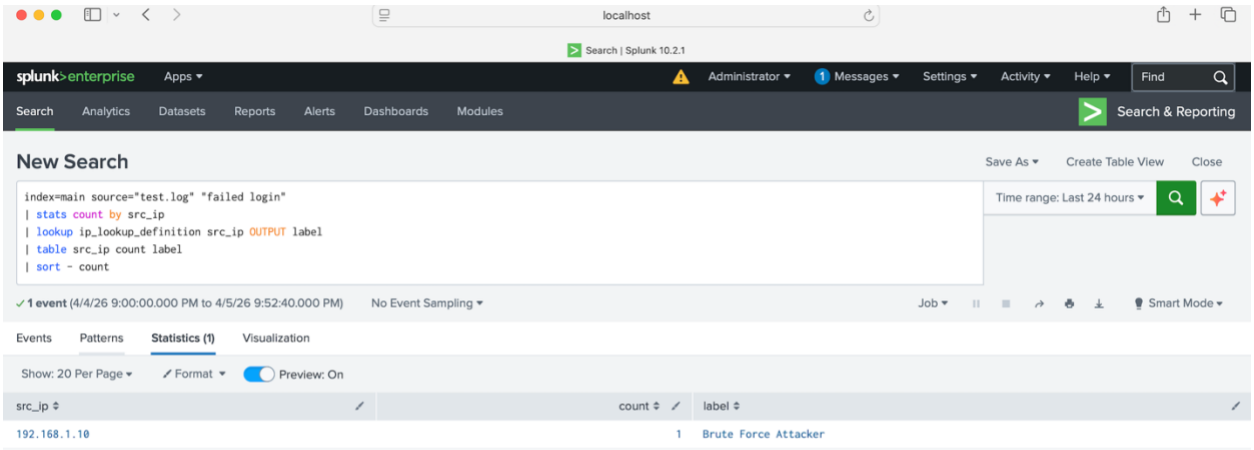
index=main source="/Users/**xxxxxxxxxxxxx**/test.log" "failed login"
| stats count by src_ip
| lookup ip_lookup_definition src_ip OUTPUT label
| table src_ip count label
| sort – count

In this step, the lookup definition was applied within the Splunk search using the `lookup` command to enrich the query results, which successfully matched source IP addresses with the external lookup table and produced labeled outputs such as *"Brute Force Detection – Failed Logins by IP"* and other classifications, thereby adding meaningful context to previously raw log data.



In this step, the lookup was successfully applied to the search results by using the `lookup` command in Splunk, which matched the `src_ip` field from the indexed log data with the corresponding entries in the uploaded CSV lookup table. After correcting the query to use the exact indexed source value (`test.log`), Splunk returned enriched results that included the original source IP address, the event count, and the descriptive label *"Brute Force Attacker."* This demonstrates how raw log data can be enhanced with external contextual information, allowing analysts to interpret suspicious activity more effectively and transform simple event data into more meaningful security intelligence.

```
index=main source="test.log" "failed login"
| stats count by src_ip
| lookup ip_lookup_definition src_ip OUTPUT label
| table src_ip count label
| sort – count
```



This step demonstrated that Splunk can search indexed log data, analyze and count suspicious activity by source IP address, correlate those IPs with an external lookup table, and present the results with meaningful classifications, which represents the core concept of data enrichment. In this case, the results showed the IP address *192.168.1.10* with a count of 1 and the label *"Brute Force Attacker,"* confirming that the lookup table was successfully matched and applied to the log data. A CSV lookup file was created, uploaded into Splunk, connected through a lookup definition, applied within an SPL query, and verified through enriched output. This demonstrates how Splunk enhances raw security events with contextual information, improving an analyst's ability to identify and interpret suspicious behavior. With these steps completed, the workflow now includes a detection query, report, dashboard, alert, and lookup enrichment, forming a complete SOC-style monitoring and analysis process.